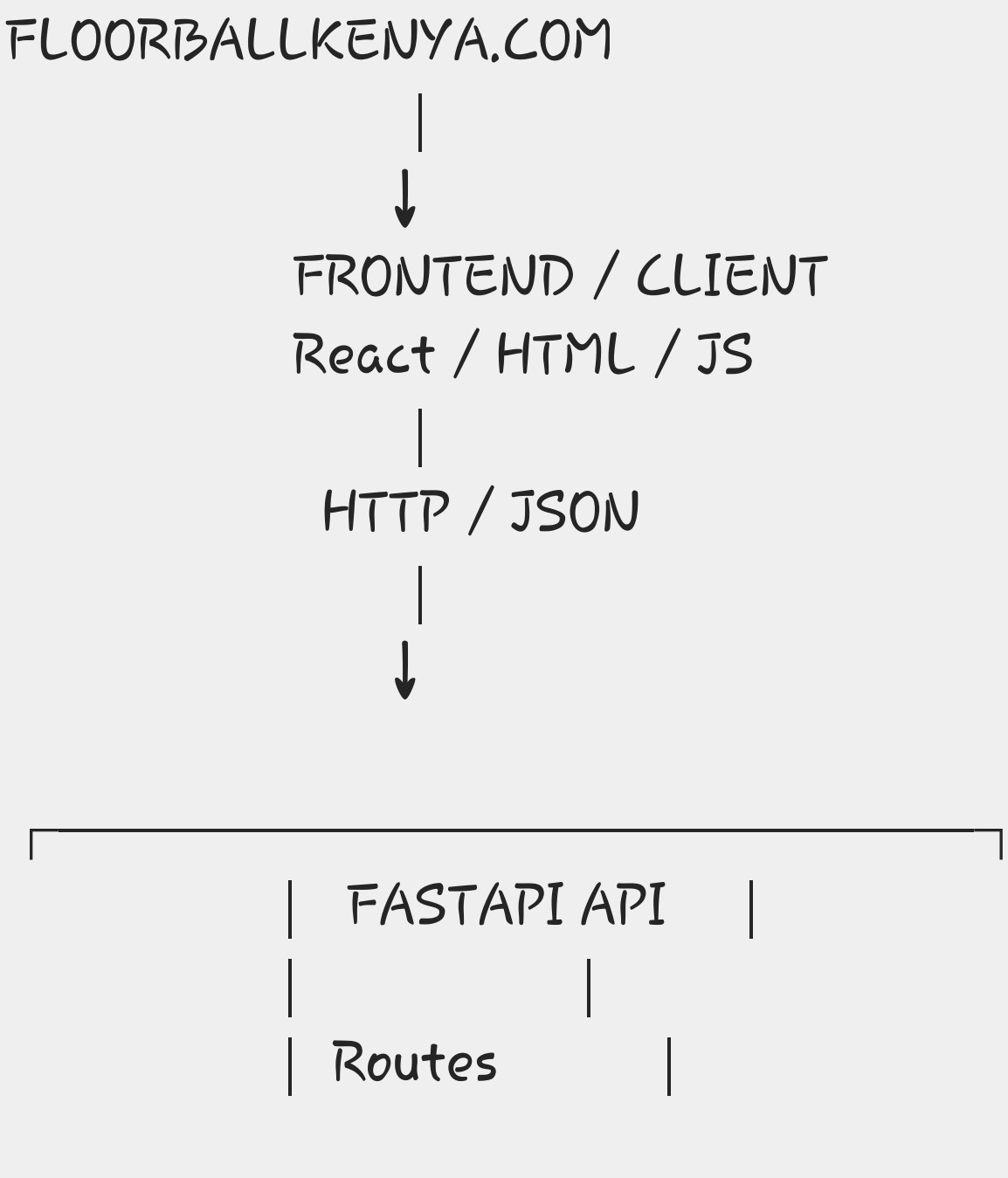
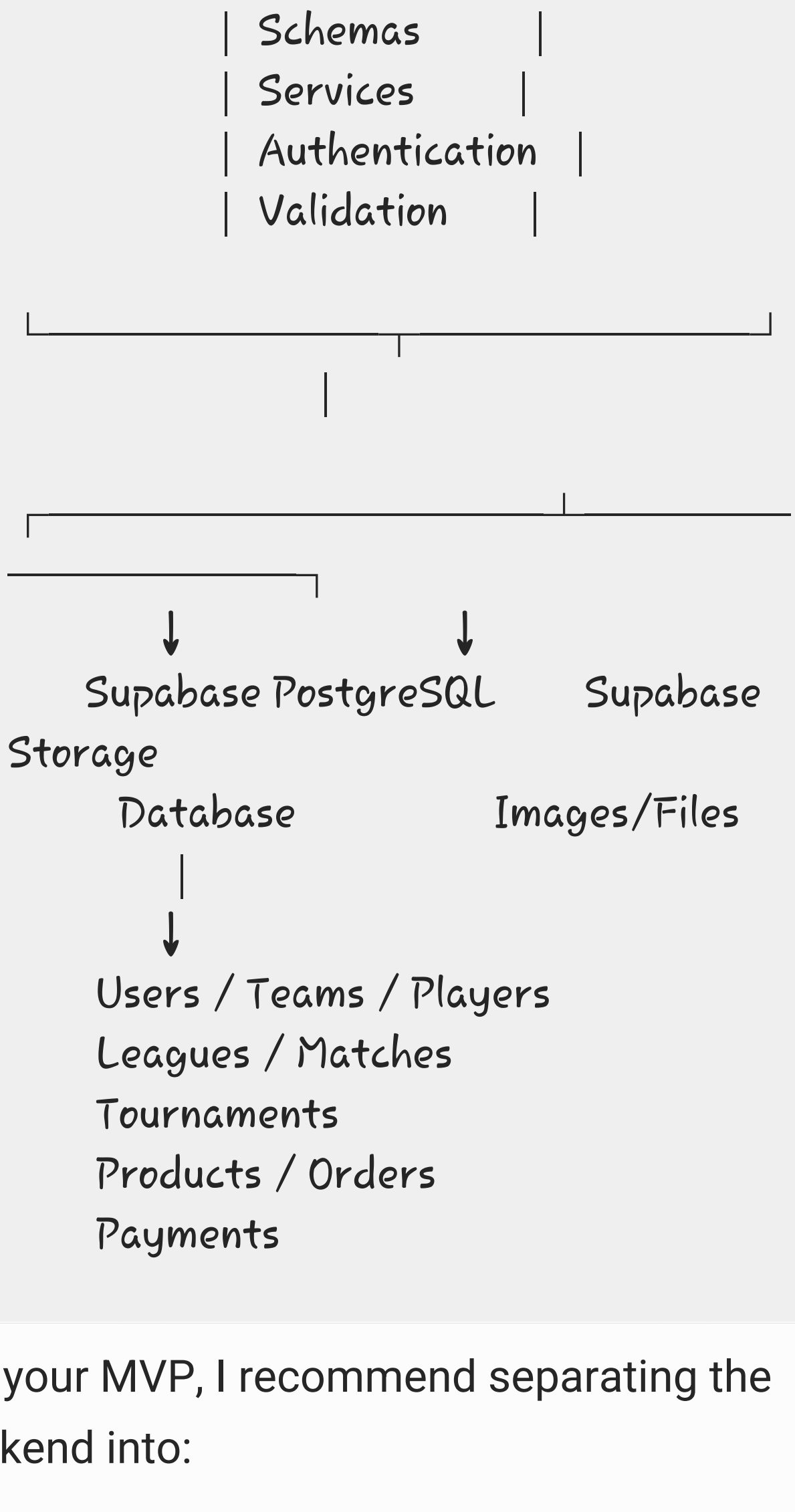
1. Recommended MVP Backend Architecture

text





For your MVP, I recommend separating the backend into:

```text
API Layer
  ↓
Authentication & Authorization
  ↓
Business Logic / Services
  ↓
Supabase Database
```

This will make your project much easier to explain during your defense.

---

2. Suggested FastAPI Project Structure

I would structure the backend like this:

text

```
floorballkenya-backend/
 │
 ├──── app/
 │     ├──── main.py
 │     │
 │     ├──── core/
 │     │     ├──── config.py
 │     │     ├──── security.py
 │     │     └──── dependencies.py
 │     │
 │     ├──── routers/
 │     │     ├──── auth.py
 │     │     ├──── users.py
 │     │     ├──── teams.py
 │     │     ├──── players.py
 │     │     ├──── leagues.py
 │     │     ├──── matches.py
 │     │     ├──── tournaments.py
 │     │     ├──── products.py
 │     │     ├──── vendors.py
 │     │     ├──── cart.py
 │     │     ├──── orders.py
```

```
│   │   ├── payments.py
│   │   ├── news.py
│   │   └── admin.py
│   │
│   ├── schemas/
│   │   ├── auth.py
│   │   ├── user.py
│   │   ├── team.py
│   │   ├── player.py
│   │   ├── league.py
│   │   ├── match.py
│   │   ├── tournament.py
│   │   ├── product.py
│   │   ├── order.py
│   │   └── payment.py
│   │
│   ├── services/
│   │   ├── standings.py
│   │   ├── statistics.py
│   │   ├── payments.py
│   │   └── commissions.py
│   │
│   └── database/
│       └── supabase.py
│
```

```
├──── .env
├──── requirements.txt
├──── Dockerfile
└──── README.md
```

This is considerably better than putting everything inside main.py.

---

3. Main Backend Modules

Your MVP backend should have these modules:

Module	Purpose
Authentication	Login, registration and JWT/ session handling

Users	Manage user profiles and roles
Teams	Manage teams
Players	Manage players
Leagues	Manage leagues and seasons
Matches	Fixtures and match results
Statistics	Calculate player/team statistics
Tournaments	Tournament registration

Marketplace	Products and vendors
Cart	Shopping cart
Orders	Customer orders
Payments	M-Pesa transactions
News	Announcements
Admin	Administrative operations

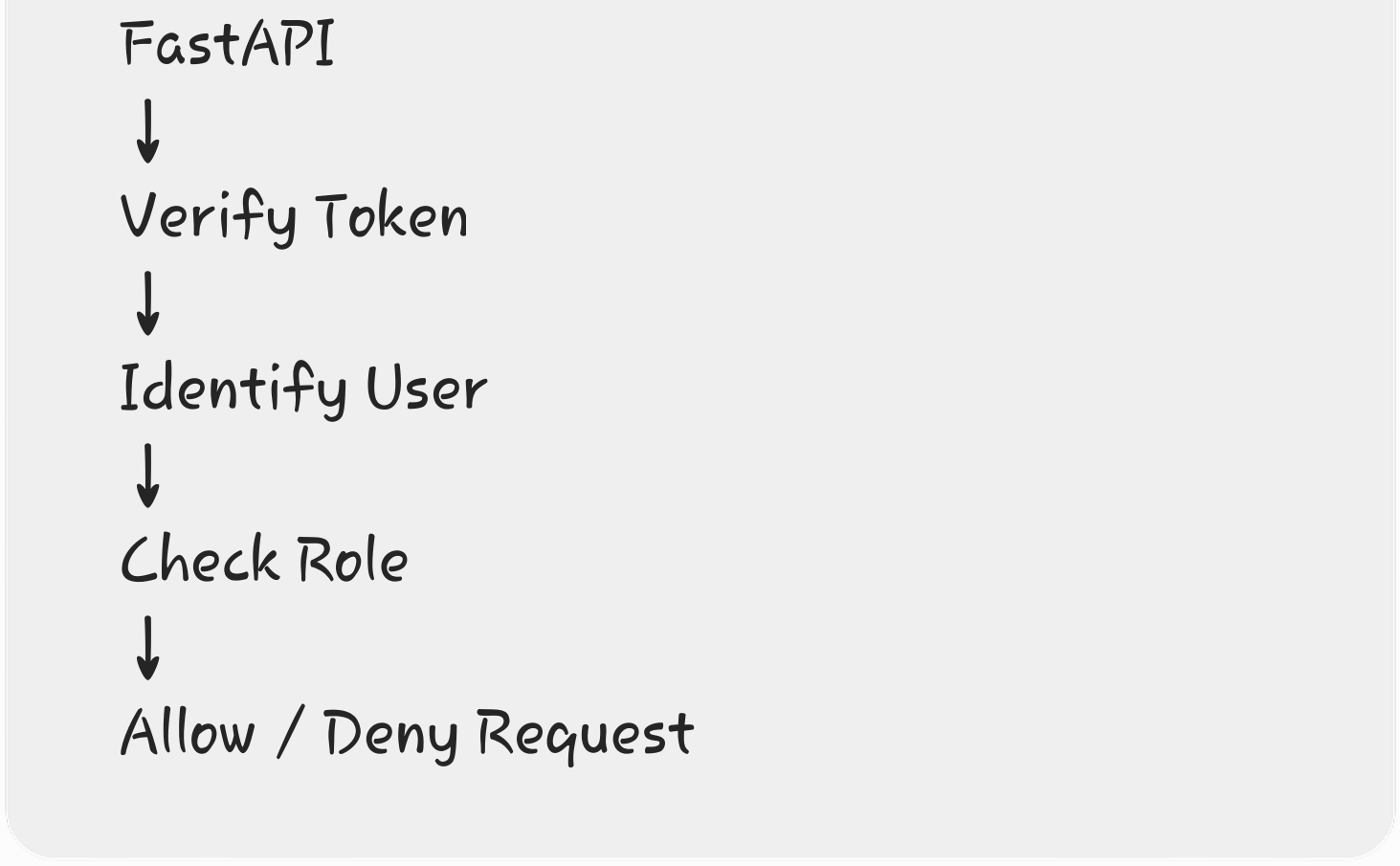
---

4. Authentication

You need authentication before building most other modules.
I recommend using **Supabase Auth** for the MVP rather than creating your own password authentication system.
Your FastAPI backend can verify the Supabase-issued access token and then determine the user's role.

Authentication flow

```text
User
↓
Login
↓
Supabase Auth
↓
Access Token
↓
Frontend
↓
```



---

5. Authentication Endpoints

POST /auth/register

Purpose

Create a new user account.

```text
POST /auth/register
```

Request:

```json
{
  "email": "player@example.com",
  "password": "password123",
  "full_name": "John Kamau",
  "phone": "0712345678",
  "role": "player"
}
```

POST /auth/login

Purpose

Authenticate a user.

```text
POST /auth/login
```

Returns an access token/session information.

GET /auth/me

Purpose

Return the currently authenticated user's information.

```text
GET /auth/me
```

POST /auth/logout

Purpose

Log the user out / invalidate the current session where applicable.

```text
POST /auth/logout
```

---

6. User Endpoints

GET /users/{user_id}

Get a user's profile.

PUT /users/{user_id}

Update a user's profile.

GET /users

Administrator-only list of users.

PATCH /users/{user_id}/status

Activate/deactivate a user.

---

7. Team Backend

Teams are one of your most important entities.

Team endpoints

Endpoint	Purpose
POST /teams	Create/register a team
GET /teams	Get all teams
GET /teams/{team_id}	Get one team
PUT /teams/{team_id}	Update team
DELETE /teams/{team_id}	Delete/deactivate team
POST /teams/{team_id}/players	Add player to team
DELETE /teams/{team_id}/players/	Remove player

{player_id}	
GET /teams/{team_id}/players	Get team roster
GET /teams/{team_id}/matches	Get team's fixtures/results
GET /teams/{team_id}/statistics	Get team statistics

---

8. Player Backend

Player endpoints

Endpoint	Purpose

POST /players	Register player
GET /players	List players
GET /players/{player_id}	Get player profile
PUT /players/{player_id}	Update player
DELETE /players/{player_id}	Deactivate player
GET /players/{player_id}/statistics	Get player statistics
GET /players/{player_id}/matches	Get player's matches
PATCH /players/{player_id}/team	Assign/change team

---

9. League Backend

The league module should manage seasons and participating teams.

Endpoints

Endpoint	Purpose
POST /leagues	Create league
GET /leagues	List leagues
GET /leagues/{league_id}	Get league details
PUT /leagues/{league_id}	Update league
DELETE /leagues/	Deactivate league

{league_id}	
POST /leagues/{league_id}/teams	Add team to league
DELETE /leagues/{league_id}/teams/{team_id}	Remove team
GET /leagues/{league_id}/teams	Get participating teams
GET /leagues/{league_id}/standings	Get league table
GET /leagues/{league_id}/fixtures	Get league fixtures
GET /leagues/{league_id}/results	Get league results
GET /leagues/{league_id}/statistics	Get league statistics

---

10. Match & Fixture Backend

This is where your system becomes interesting.

Endpoints

Endpoint	Purpose
POST /matches	Create fixture
GET /matches	List matches
GET /matches/{match_id}	Get match
PUT /matches/{match_id}	Update fixture
DELETE /matches/	Cancel fixture

{match_id}	
POST /matches/{match_id}/result	Record match result
GET /matches/{match_id}/result	View result

---

11. Recording a Match Result

This endpoint is extremely important:

```text
POST /matches/{match_id}/result
```

Example request:

json


```
{
 "home_score": 5,
 "away_score": 3,
 "goal_scorers": [
  {
   "player_id": "123",
   "goals": 3
  },
  {
   "player_id": "456",
   "goals": 2
  }
 ]
}
```


When this happens, your backend should:

text

```
Record Result
     ↓
Update Match
```

```
        ↓
Update Player Statistics
        ↓
Update Team Statistics
        ↓
Recalculate League Standings
```

That business logic should live in a service such as:

```text
services/statistics.py
services/standings.py
```

rather than directly inside the route.

---

12. Standings Logic

Your backend should automatically calculate

For example:

```text
Win  = 3 points
Draw = 1 point
Loss = 0 points
```

After every completed match:

```text
POST /matches/{id}/result
```

calls:

```text
update_team_statistics()
        ↓
update_player_statistics()
        ↓
recalculate_standings()
```

Then:

```text
GET /leagues/{id}/standings
```

returns the current table.

---

13. Tournament Backend

Endpoints

Endpoint	Purpose
POST /tournaments	Create tournament
GET /tournaments	List tournaments
GET /tournaments/	Tournament details

{id}	
PUT /tournaments/{id}	Update tournament
DELETE /tournaments/{id}	Deactivate tournament
POST /tournaments/{id}/register	Register team
GET /tournaments/{id}/teams	View registered teams
PATCH /tournaments/{id}/registrations/{registration_id}	Approve/reject registration
GET /tournaments/{id}/matches	View tournament matches

---

14. Vendor Backend

Endpoints

Endpoint	Purpose
POST /vendors	Apply as vendor
GET /vendors	List vendors
GET /vendors/{id}	Vendor profile
PUT /vendors/{id}	Update vendor
PATCH /vendors/{id}/approve	Approve vendor
PATCH /vendors/{id}/suspend	Suspend vendor
GET /vendors/{id}/	Vendor products

products	
GET /vendors/{id}/orders	Vendor orders

---

15. Product Backend

Endpoints

Endpoint	Purpose
POST /products	Add product
GET /products	List products
GET /products/{id}	Product details
PUT /products/{id}	Update product

DELETE /products/{id}	Remove product
PATCH /products/{id}/stock	Update stock
GET /products/search	Search products

You should support filtering:

```text
GET /products?category=sticks
GET /products?min_price=1000&max_price=10000
GET /products?vendor_id=123
```

---

16. Shopping Cart

For the MVP:

Endpoints

```text
GET /cart
```

Get current user's cart.

```text
POST /cart/items
```

Add product.

```text
PATCH /cart/items/{item_id}
```

Change quantity.

```text
DELETE /cart/items/{item_id}
```

Remove product.

```text
DELETE /cart
```

Clear cart.

---

17. Orders

Endpoints

Endpoint	Purpose

POST /orders	Create order from cart
GET /orders	Get user's orders
GET /orders/{id}	Get order details
PATCH /orders/{id}/status	Update order status
POST /orders/{id}/cancel	Cancel order

Order lifecycle:

```text
Pending
  ↓
Paid
  ↓
Processing
  ↓
Delivered
```

```
↓
Completed
```

---

18. M-Pesa Backend

For the MVP, keep this module simple.

Endpoints

```text
POST /payments/mpesa/initiate
```

Initiates an M-Pesa payment.

```text
POST /payments/mpesa/callback
```

Receives the M-Pesa callback.

```text
GET /payments/{transaction_id}
```

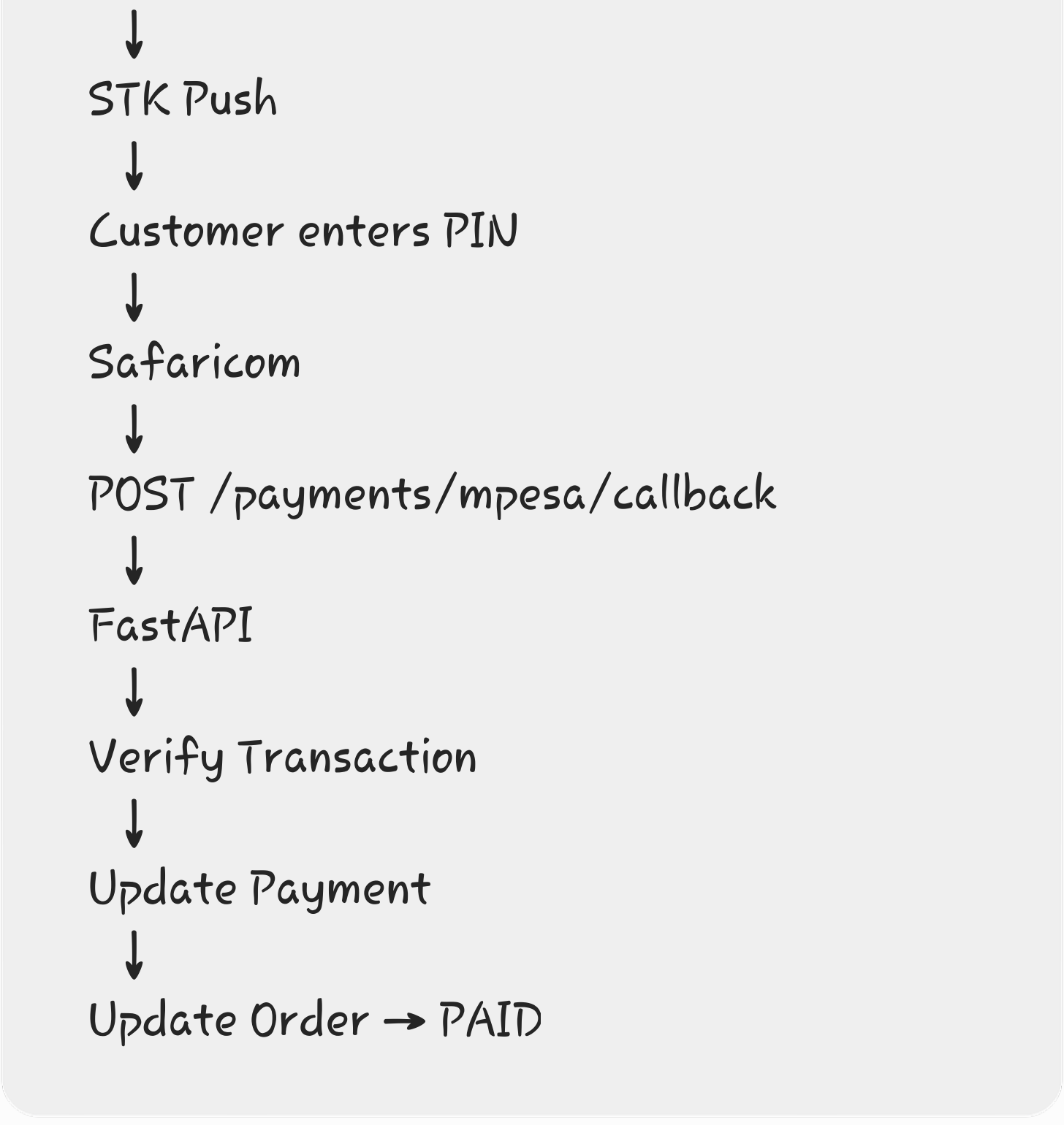
Returns transaction status.

---

19. M-Pesa Flow

Your architecture should look like:

```text
Customer
  ↓
Checkout
  ↓
POST /payments/mpesa/initiate
  ↓
Safaricom M-Pesa
```



Do not trust the frontend to tell your API that payment succeeded. The backend should use the payment provider's callback/result and server-side verification.

For your university MVP, you can demonstrate this with a sandbox/test environment before

production deployment.

---

20. Commission Backend

Endpoint

```text
GET /commissions
```

Administrator can view commissions.

```text
GET /vendors/{vendor_id}/earnings
```

Vendor can view earnings.

When an order is successfully paid:

```text
Order
↓
Calculate Total
↓
Calculate Commission
↓
Store Commission
↓
Store Vendor Earnings
```

For example:

```text
Product = KSh 5,000

10% Commission = KSh 500

Vendor Earnings = KSh 4,500
```

For the MVP, calculate and record the

rather than implementing actual automated escrow/payout settlement.

---

21. News Backend

Endpoints

```text
POST /news
```

Create news article.

```text
GET /news
```

Get published articles.

```text
GET /news/{id}
```

View article.

```text
PUT /news/{id}
```

Edit article.

```text
DELETE /news/{id}
```

Delete article.

---

22. Admin Backend

You don't need a separate endpoint for every dashboard card.
Instead, create administrative endpoints such as:

```text
GET /admin/dashboard
```

Returns:

```json
{
  "users": 120,
  "teams": 15,
  "players": 210,
  "leagues": 3,
  "tournaments": 5,
  "vendors": 8,
  "products": 72,
  "orders": 45
}
```

Other admin endpoints:

```text
GET /admin/users
GET /admin/vendors
GET /admin/orders
GET /admin/payments
GET /admin/reports
```

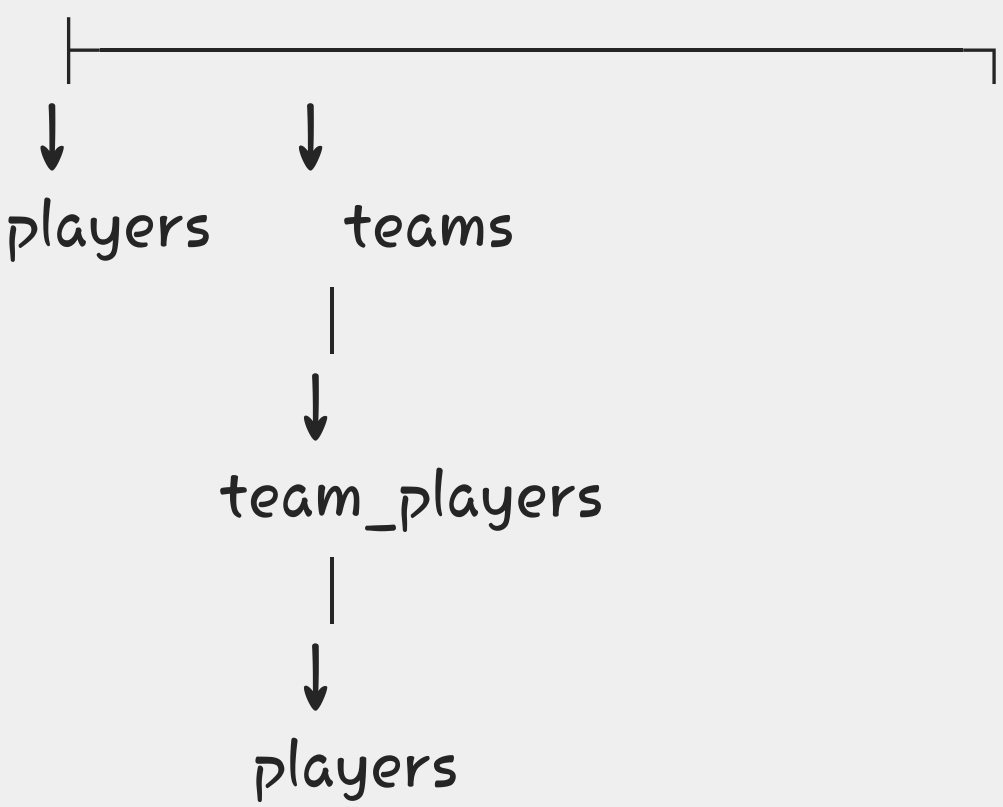
---

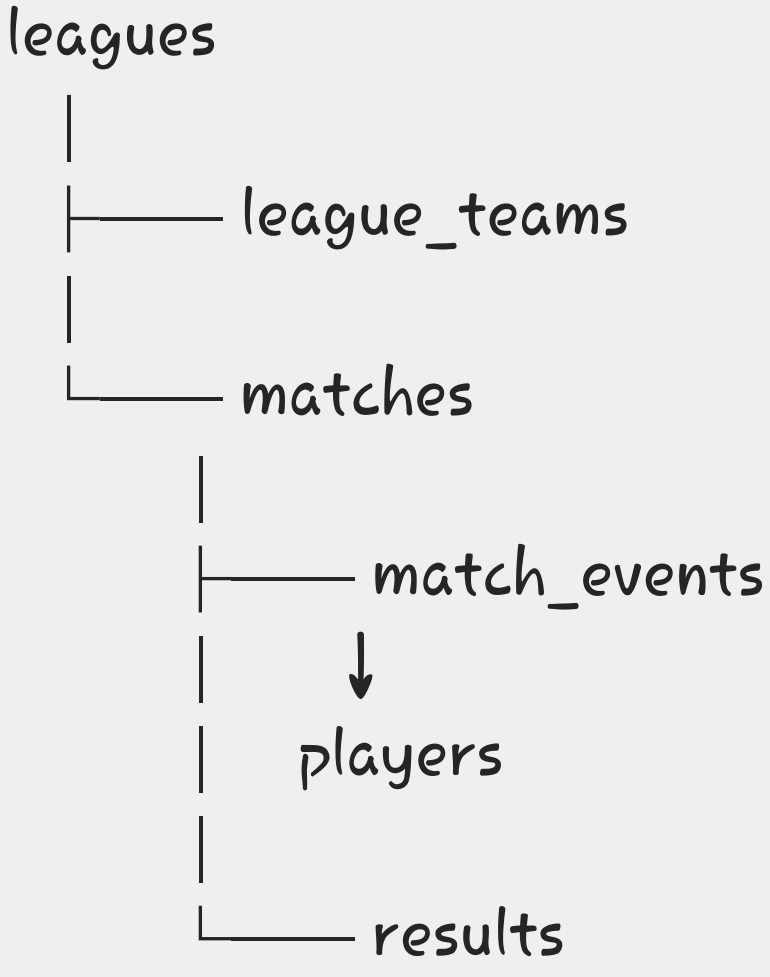
23. Database Design

Supabase PostgreSQL should contain your core entities.

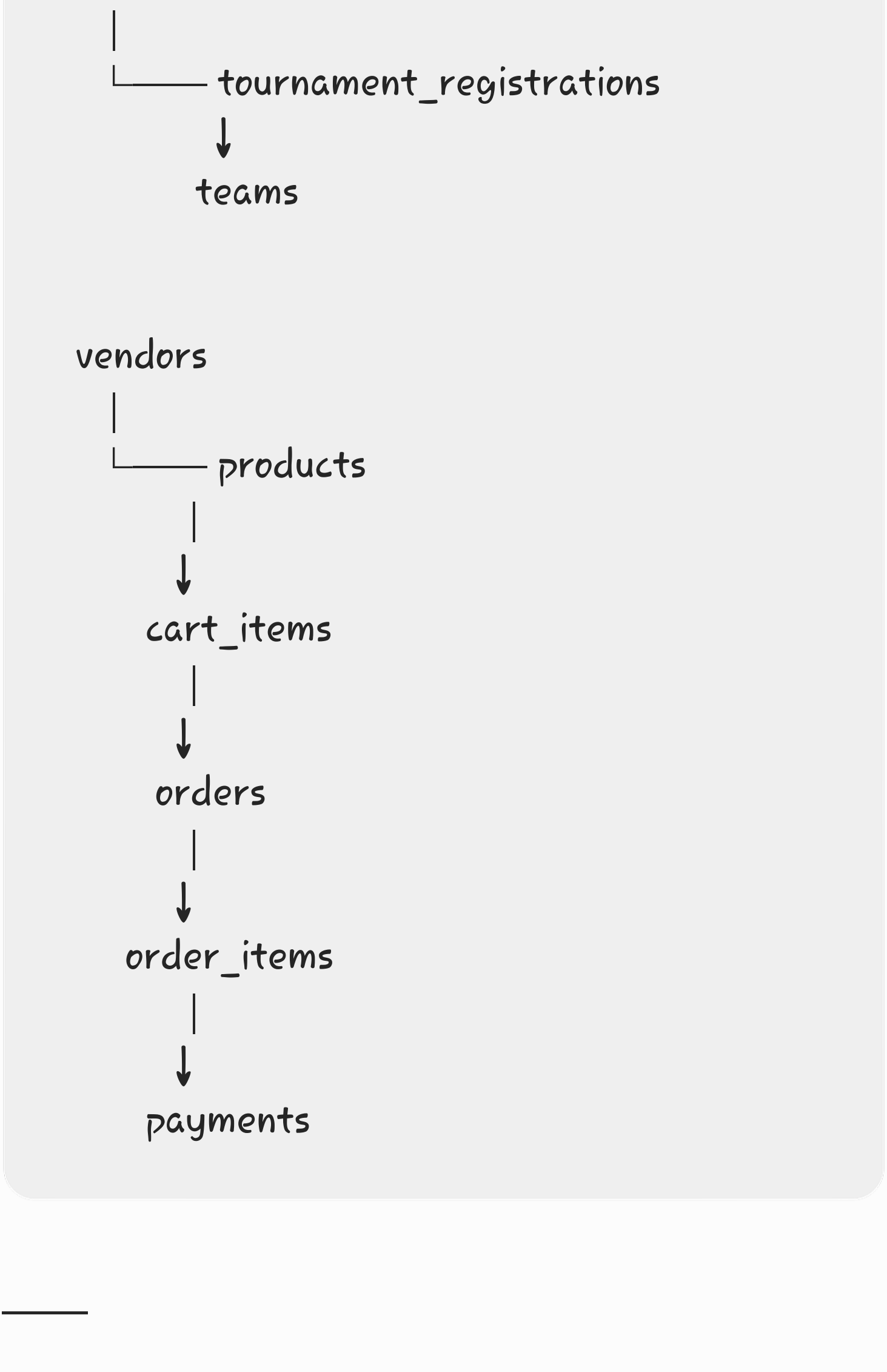
A simplified MVP database could look like:

```text
users
  |
```





tournaments


tournament_registrations
teams
vendors
products
cart_items
orders
order_items
payments

24. Important Tables

For the MVP, I would start with roughly these tables:

Authentication/Profile

```text
profiles
```

Stores:

- id
- full_name
- email
- phone
- role
- profile_image
- created_at

Supabase Auth handles the actual authentication credentials.

Sports

```text
teams
players
team_players
leagues
seasons
league_teams
matches
match_events
player_statistics
team_statistics
```

Competitions

```text
tournaments
```

```
tournament_registrations
```

Marketplace

```text
vendors
products
product_categories
cart_items
orders
order_items
```

Payments

```text
payments
commissions
```

Content

```text
news
```

---

25. Role-Based Authorization

This is very important.

Don't just authenticate users. **Authorize them.**

For example:

```text
ADMIN
  ├──── Manage everything
  │
TEAM_MANAGER
  ├──── Manage own team
  ├──── Manage own players
  └──── Register team
```

```
  |
PLAYER
  ├──── View own profile
  └──── View statistics
  |
VENDOR
  ├──── Manage own products
  └──── Manage own orders
  |
CUSTOMER
  ├──── Cart
  ├──── Orders
  └──── Payments
```

So:

```text
DELETE /teams/123
```

should not mean every logged-in user can delete Team 123.

The backend should check:

```text
Is user authenticated?
        ↓
What is user's role?
        ↓
Does user own/manage this resource?
        ↓
ALLOW / DENY
```

---

26. API Endpoint Summary

For your documentation, you can summarize the MVP API as:

Module	Main Endpoints

Auth	/auth/register, /auth/login, /auth/me, /auth/logout
Users	/users, /users/{id}
Teams	/teams, /teams/{id}, /teams/{id}/players
Players	/players, /players/{id}, /players/{id}/statistics
Leagues	/leagues, /leagues/{id}, /leagues/

	{id}/standings
Matches	/matches, /matches/{id}, /matches/{id}/result
Tournaments	/tournaments, /tournaments/{id}/register
Vendors	/vendors, /vendors/{id}
Products	/products, /products/{id}
Cart	/cart, /cart/items
Orders	/orders, /orders/{id}

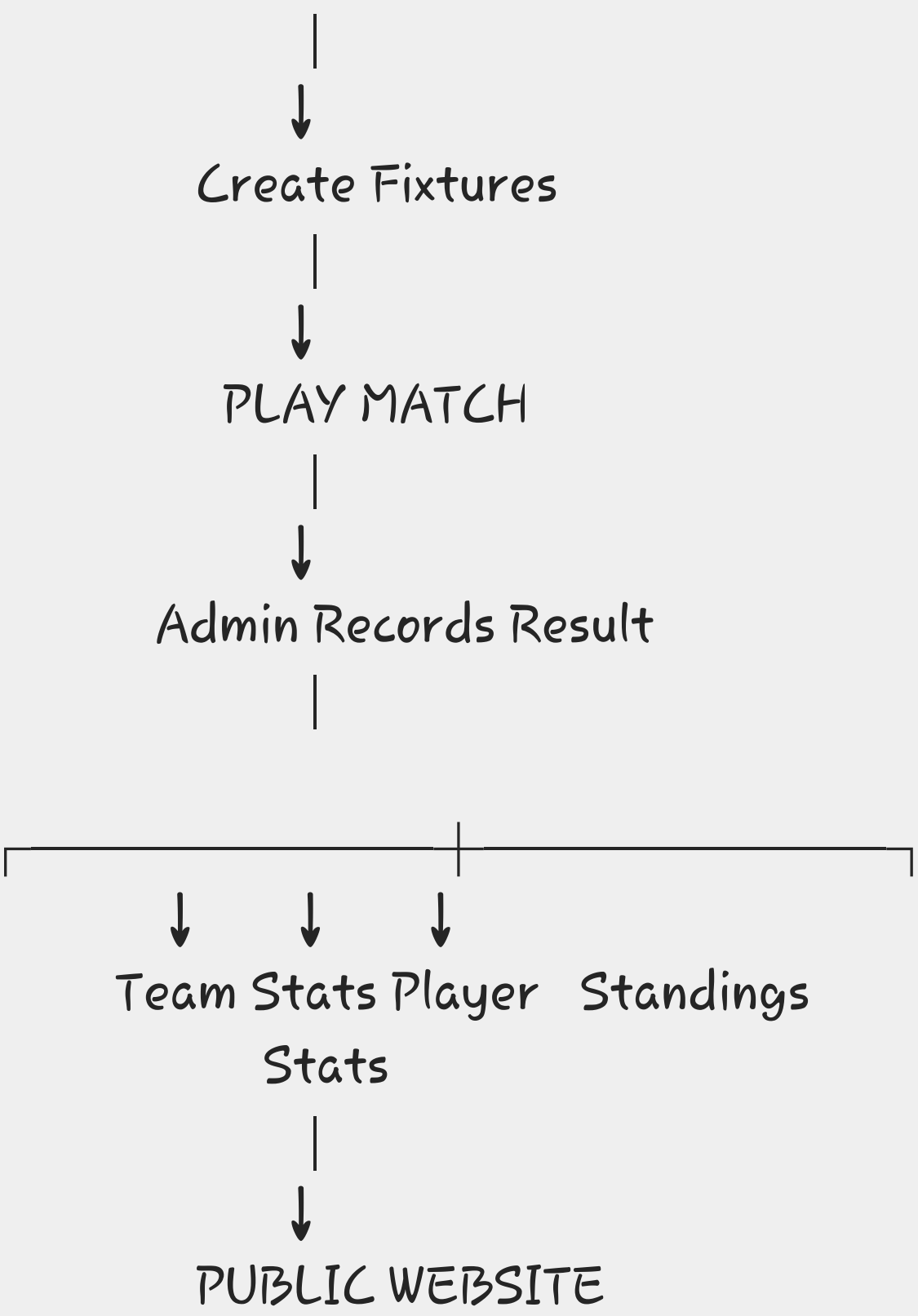
Payments	/payments/mpesa/initiate, /payments/mpesa/callback
Commissions	/commissions, /vendors/{id}/earnings
News	/news, /news/{id}
Admin	/admin/dashboard, /admin/users, /admin/reports

27. The Most Important MVP Backend Flow

If I were building your project, I would make this the central demonstration flow:

text

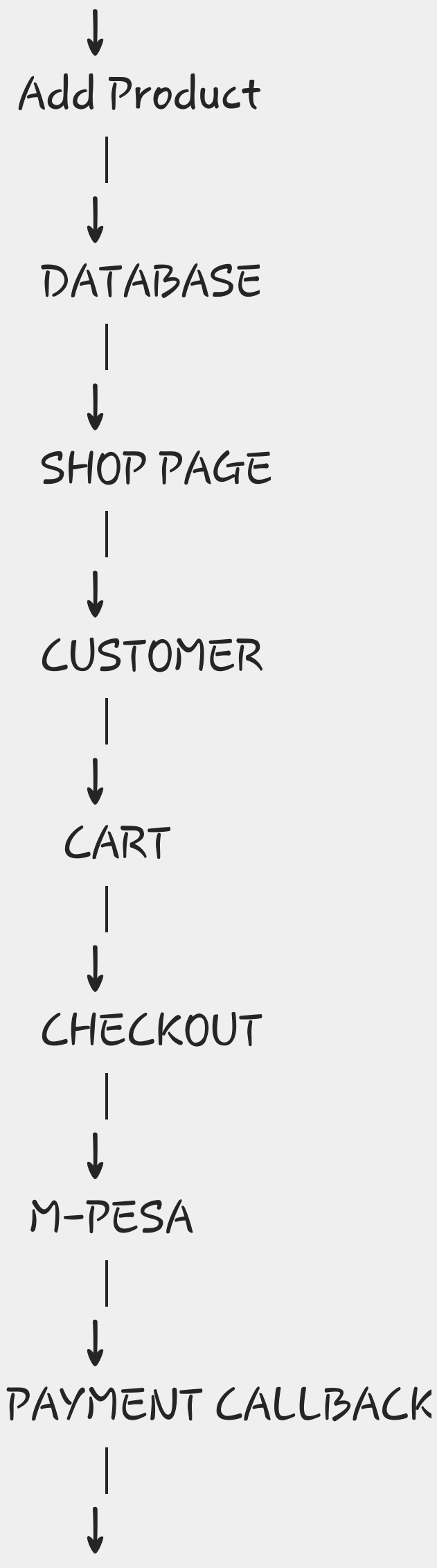


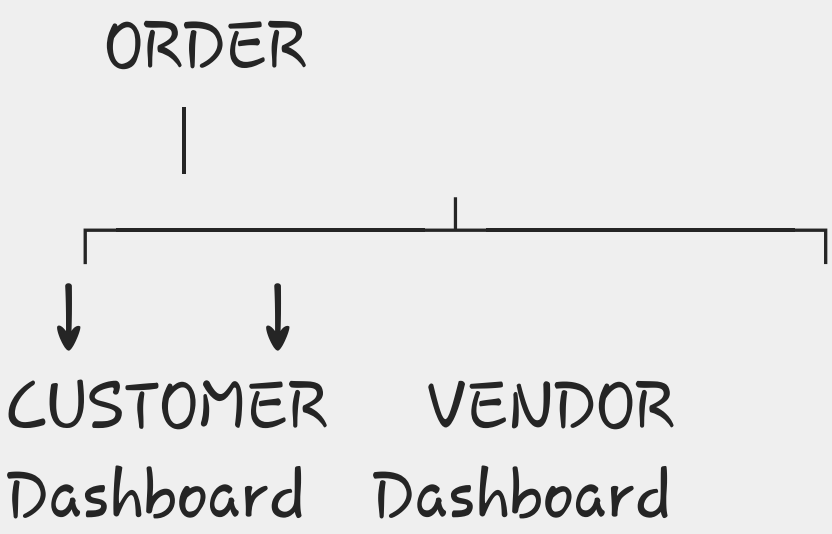


Then your second major demonstration flow is:

text

```
VENDOR
```


Add Product
DATABASE
SHOP PAGE
CUSTOMER
CART
CHECKOUT
M-PESA
PAYMENT CALLBACK


ORDER
CUSTOMER
Dashboard
VENDOR
Dashboard